National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Artificial Intelligence & Data Science

DS201: Programming for AI

Class: BSAI

Lab 4: Building a Leakage-Safe ML Pipeline with Pandas and Scikit-learn

Date: 17-02-2026

Time: 10:00 to 13:00

Instructor: Dr. Mehwish Fatima

Lab Engineer: Mr. Hafiz Arslan Ramzan

Muhammad Sikandar Hussain 502808

CLO-1 (C4 – Analyze)

Analyze and develop complete ML/DL pipelines by performing data preprocessing, feature engineering, model training, evaluation, testing, and serialization.

CLO-4 (P4 – Mechanism, Psychomotor)

Implement, containerize, and operationalize AI systems using Docker and foundational MLOps practices to produce deployable AI service

Objectives of this lab:

By the end of this lab, students will be able to:

- Develop complete leakage-safe ML pipelines (classification + regression)
- Perform proper train/test splitting and prevent data leakage
- Implement preprocessing using train-only fitting
- Compare multiple ML models using appropriate metrics
- Build sklearn Pipelines for reproducibility
- Serialize trained pipelines for deployment readiness
- Produce script-based, reproducible AI workflows

Lab Tasks

Task 1:

Classification Task: Build a classification model to predict whether a passenger survived. **Target variable:** Survived (0 = No, 1 = Yes)

Dataset

Use the Titanic dataset (load from seaborn):

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("titanic")
```

Data Understanding & Proper Split

Step 1: Data Exploration

1. Inspect dataset shape
2. Display first 5 rows
3. Identify missing values
4. Identify categorical and numerical columns

```
df.head()
df.info()
df.isnull().sum()
```

Observe:

- Which columns contain missing values?
- Which columns are categorical?
- Which columns are numerical?

Step 2: Select Features

Drop irrelevant columns such as:


```
df = df.drop(columns=["alive", "embark_town", "class", "who"])
```

Separate features and target:

```
X = df.drop("survived", axis=1)
y = df["survived"]
```

Step 3: Train/Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Important Rule: *Split BEFORE preprocessing.*

Preprocessing + Model Training

Step 4: Identify Column Types

```
num_cols = X_train.select_dtypes(include=["int64", "float64"]).columns
cat_cols = X_train.select_dtypes(include=["object", "category",
"bool"]).columns
```

Step 5: Handle Missing Values (Train Only)

```
from sklearn.impute import SimpleImputer
num_imputer = SimpleImputer(strategy="mean")
cat_imputer = SimpleImputer(strategy="most_frequent")

X_train[num_cols] = num_imputer.fit_transform(X_train[num_cols])
X_test[num_cols] = num_imputer.transform(X_test[num_cols])

X_train[cat_cols] = cat_imputer.fit_transform(X_train[cat_cols])
X_test[cat_cols] = cat_imputer.transform(X_test[cat_cols])
```

Step 6: Encoding & Scaling

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
scaler = StandardScaler()

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

X_train = pd.get_dummies(X_train, columns=cat_cols, drop_first=True)
X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True)

X_test = X_test.reindex(columns=X_train.columns,

fill_value=0)
```

Model Training

Train 3 different models:

Model 1: Logistic Regression


```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
```

Model 2: Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
```

Model 3: Support Vector Machine

```
from sklearn.svm import SVC
svm_model = SVC()
svm_model.fit(X_train, y_train)
```

Evaluation

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

models = {
    "Logistic Regression": log_model,
    "Random Forest": rf_model,
    "SVM": svm_model
}

for name, model in models.items():
    train_preds = model.predict(X_train)
    test_preds = model.predict(X_test)

    print(f"\n{name}")
    print("Train Accuracy:", accuracy_score(y_train, train_preds))
    print("Test Accuracy:", accuracy_score(y_test, test_preds))
    print("Precision:", precision_score(y_test, test_preds))
    print("Recall:", recall_score(y_test, test_preds))
    print("F1 Score:", f1_score(y_test, test_preds))

#For imbalanced dataset
precision_score(y_test, test_preds, average='weighted')
recall_score(y_test, test_preds, average='weighted')
f1_score(y_test, test_preds, average='weighted')
```

Record Results

Model	Train Accuracy	Test Accuracy	F1	Precision	Recall
Logistic Regression	0.8370786	0.8156424	0.7755102	0.7808219	0.77027027
Random Forest	0.9845505	0.7932960	0.7448275	0.7605636	0.72972972

SVM	0.8356741	0.8156424	0.765957	0.8059701	0.72972972

Or

```
from sklearn.metrics import classification_report
for name, model in models.items():
    print(f"\n{name}")
    preds = model.predict(X_test)
    print(classification_report(y_test, preds))
```

Introduce Leakage

Now intentionally do this WRONG:

1. Impute entire dataset
2. Scale entire dataset
3. Then split

Compare test performance.

Observe:

1. Did test performance increase?
2. Why?
3. What leaked?

- Mean and standard deviation from test set influenced scaling parameters
- Most frequent categories from test set influenced imputation
- Model indirectly "saw" test data during preprocessing

Scikit-learn Pipeline (Abstraction)

Step 1: ColumnTransformer

```
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)])
```

Step 2: Full Pipeline

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=1000))])
```

Step 3: Train & Evaluate

```
pipeline.fit(X_train, y_train)
preds = pipeline.predict(X_test)
print("Pipeline Accuracy:", accuracy_score(y_test, preds))
```


Step 4: Model Serialization (For Deployment Readiness)

Once the pipeline is trained, save it for reuse:

```
import joblib
joblib.dump(pipeline, "titanic_pipeline.joblib")
```

This file now contains:

- Preprocessing steps
- Learned scaling parameters
- Encoding mappings
- Trained model weights

To load later:

```
loaded_pipeline = joblib.load("titanic_pipeline.joblib")
preds = loaded_pipeline.predict(X_test)
```

Model Comparison

Compare:

- Manual workflow
- Leakage workflow
- Proper Pipeline workflow

Discuss:

- Which is safest?
- Which is most reproducible?
- Why pipelines prevent leakage?

Final Reflection Questions

Answer in your report:

1. Why must we split before preprocessing?

Preprocessing parameters (mean, std, most frequent value) must be learned only from training data. If we preprocess before splitting, test set statistics influence these parameters, causing data leakage and overly optimistic performance estimates.

2. What is data leakage?

Data leakage occurs when information from outside the training dataset is used to create the model. This includes letting test set statistics influence preprocessing or feature engineering, leading to unrealistic performance metrics that won't generalize to new data.

3. Why can small leakage significantly inflate performance?

Even small amounts of leaked information can help the model "cheat" by incorporating patterns from the test set. This makes the model appear more accurate than it actually is, leading to poor real-world performance where such information isn't available.

4. What is the role of `fit()` vs `transform()`?

`fit()`: Learns parameters from the training data (e.g., mean, std for scaling)
- `transform()`: Applies learned parameters to transform data
- `fit_transform()`: Combines both steps (only use on training data)
- Test data should only use `transform()` with parameters learned from training

5. Why is Pipeline safer than manual steps?

- Ensures preprocessing and training steps are always applied in correct order
- Prevents accidentally fitting preprocessors on test data
- Encapsulates entire workflow for reproducibility
- Makes deployment easier by packaging everything together
- Reduces human error in multi-step workflows

```
task 1: classification - titanic survival prediction
--------------------------------------------------

dataset shape: (891, 15)

first 5 rows:
   survived  pclass     sex   age  sibsp  parch     fare  ...  class    who adult_male  deck  embark_town alive  alone
0         0       3    male  22.0      1      0   7.2500  ...  Third    man       True   NaN  Southampton    no  False
1         1       1  female  38.0      1      0  71.2833  ...  First  woman      False     C    Cherbourg   yes  False
2         1       3  female  26.0      0      0   7.9250  ...  Third  woman      False   NaN  Southampton   yes   True
3         1       1  female  35.0      1      0  53.1000  ...  First  woman      False     C  Southampton   yes  False
4         0       3    male  35.0      0      0   8.0500  ...  Third    man       True   NaN  Southampton    no   True

[5 rows x 15 columns]
```

Outputs:

```
dataset info:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    str
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    str
 8   class        891 non-null    category
 9   who          891 non-null    str
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    str
 13  alive        891 non-null    str
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

```
missing values:
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64

numerical columns: ['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare']
categorical columns: ['sex', 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town', 'alive', 'alone']

data split: train=712, test=179
```

```
training models - manual preprocessing
--------------------------------------------------

Logistic Regression
  train accuracy: 0.8371
  test accuracy:  0.8156
  precision:      0.7808
  recall:         0.7703
  f1 score:       0.7755

Random Forest
  train accuracy: 0.9846
  test accuracy:  0.7933
  precision:      0.7606
  recall:         0.7297
  f1 score:       0.7448

SVM
  train accuracy: 0.8357
  test accuracy:  0.8156
  precision:      0.8060
  recall:         0.7297
  f1 score:       0.7660
```

```
summary table:
              Model  Train Accuracy  Test Accuracy  Precision  Recall  F1 Score
Logistic Regression        0.837079       0.815642   0.780822 0.77027  0.775510
      Random Forest        0.984551       0.793296   0.760563 0.72973  0.744828
                SVM        0.835674       0.815642   0.805970 0.72973  0.765957
```

```
demonstrating data leakage (incorrect approach)
--------------------------------------------------

logistic regression with data leakage:
  test accuracy: 0.8156
  f1 score:      0.7755

problem: preprocessing was done on entire dataset before split!
this allows test data statistics to leak into training

sklearn pipeline approach
--------------------------------------------------

pipeline logistic regression:
  train accuracy: 0.8371
  test accuracy:  0.8156
  f1 score:       0.7755

pipeline saved to: titanic_pipeline.joblib
```

Task 2:

Regression Task: Build regression models to predict house prices.
Target variable:price (continuous)

Dataset

Use California Housing dataset (built-in sklearn, no dependency issues):

```
from sklearn.datasets import fetch_california_housing
import pandas as pd

data = fetch_california_housing(as_frame=True)
```



```
df = data.frame
```

Target:

```
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]
```

Data Understanding & Proper Split

Step 1 — Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Reminder: Split before scaling.

Step 2 — Preprocessing

Only numeric features here.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Train Regression Models

Train two models:

Model 1: Linear Regression

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
```

Model 2: Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train) # no scaling needed for trees
```

-

Evaluate Performance

Use:

- MAE
- RMSE
- R^2

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
import numpy as np

def evaluate(model, X_test, y_test):
    preds = model.predict(X_test)
    print("MAE:", mean_absolute_error(y_test, preds))
    print("RMSE:", np.sqrt(mean_squared_error(y_test, preds)))
    print("R2:", r2_score(y_test, preds))
    print("-"*30)

print("Linear Regression")
evaluate(lr, X_test_scaled, y_test)

print("Random Forest")
evaluate(rf, X_test, y_test)
```

Pipeline Serialization

Instead of saving only the model, save preprocessing + model together.
Modify regression task slightly:

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
     ("scaler", StandardScaler()),
     ("model", LinearRegression())])

pipeline.fit(X_train, y_train)
joblib.dump(pipeline, "housing_pipeline.joblib")
```

This ensures:
`Raw input → Scaling → Prediction`
is preserved exactly.

Loading Later

```
loaded_model = joblib.load("housing_pipeline.joblib")
preds = loaded_model.predict(X_test)
```

No retraining required.

Record Results

Model	MAE	RMSE	R^2

Linear Regression	**0.533200**	**0.745581**	**0.575788**
Random Forest	**0.327543**	**0.505340**	**0.805123**

Outputs:

```
task 2: regression - california housing price prediction
--------------------------------------------------

dataset shape: (20640, 9)

first 5 rows:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude  MedHouseVal
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88    -122.23        4.526
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86    -122.22        3.585
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85    -122.24        3.521
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85    -122.25        3.413
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85    -122.25        3.422
```

```
dataset info:
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   MedInc       20640 non-null  float64
 1   HouseAge     20640 non-null  float64
 2   AveRooms     20640 non-null  float64
 3   AveBedrms    20640 non-null  float64
 4   Population   20640 non-null  float64
 5   AveOccup     20640 non-null  float64
 6   Latitude     20640 non-null  float64
 7   Longitude    20640 non-null  float64
 8   MedHouseVal  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

```
statistical summary:
              MedInc      HouseAge      AveRoom
count  20640.000000  20640.000000  20640.00000
mean       3.870671     28.639486      5.42900
std        1.899822     12.585558      2.47417
min        0.499900      1.000000      0.84615
25%        2.563400     18.000000      4.44071
50%        3.534800     29.000000      5.22912
75%        4.743250     37.000000      6.05238
max       15.000100     52.000000    141.90909

[8 rows x 9 columns]

no missing values in this dataset

data split: train=16512, test=4128
```

```
training regression models - manual preprocessing
--------------------------------------------------

model evaluation:

Linear Regression
  MAE:  0.5332
  RMSE: 0.7456
  R²:   0.5758

Random Forest
  MAE:  0.3275
  RMSE: 0.5053
  R²:   0.8051

summary table:
            Model      MAE     RMSE       R²
Linear Regression 0.533200 0.745581 0.575788
    Random Forest 0.327543 0.505340 0.805123
```

```
demonstrating data leakage (incorrect approach)
--------------------------------------------------

linear regression with data leakage:
  r2 score: 0.5758

problem: scaling was done on entire dataset before split!
test set statistics leaked into training phase

sklearn pipeline approach
--------------------------------------------------

pipeline linear regression:
  train r2: 0.6126
  test r2:  0.5758
  mae:      0.5332
  rmse:     0.7456

pipeline saved to: housing_pipeline.joblib
pipeline successfully loaded and tested
```


Reflections

1. Why do we not use accuracy in regression?

Accuracy measures exact matches (correct vs incorrect), which doesn't make sense for continuous values. Regression uses MAE (average error), RMSE (penalizes large errors), and R^2 (variance explained) instead.

2. Why is RMSE larger than MAE?

RMSE squares errors before averaging, which penalizes larger errors more heavily. After squaring, large errors dominate the metric, resulting in a higher value than MAE which treats all errors equally.

3. Why does Random Forest not require scaling?

Tree-based models make decisions using feature splits (e.g., age > 30), not distances. They are invariant to monotonic transformations like scaling. Only distance-based models (SVM, KNN, Linear) require scaling.

4. Which model performs better? Why?

Random Forest typically performs better due to its ability to capture non-linear relationships and interactions between features.

5. What happens if you scale the entire dataset before splitting?

Data leakage occurs - test set statistics (mean, std) influence the scaling parameters, leading to optimistically biased performance metrics.

Deliverables

Submit a single ZIP file: **Lab4_RollNo_Name.zip**

Required Files

1. `task1_classification.py`
2. `task2_regression.py`
3. (optional) `utils.py`
4. `titanic_pipeline.joblib`
5. `housing_pipeline.joblib`
6. `README.md`

README.md Must Include

- Dataset used and how it was loaded
- Train/test split configuration (test_size, random_state)
- Preprocessing steps and where `.fit()` happens
- Models trained
- Evaluation metrics:
    - Classification: Accuracy, Precision, Recall, F1
    - Regression: MAE, RMSE, R^2
- Results tables (copy values)
- Leakage experiment explanation
- Confirmation of model serialization
- Any assumptions or issues

Optional (Not Graded)

- `exploration.ipynb` (only for data inspection)

⚠ ⚠️Jupyter notebooks may be used **only for exploration**.

Assessment Rubrics (Total Marks: 10)

CLO-2 (7 Marks – Direct)

Complete ML Pipeline Implementation (3 Marks)

- Correct train/test split
- Train-only fitting for preprocessing
- No leakage in final workflow
- Proper sklearn Pipeline abstraction

- Serialization of pipeline using joblib

Model Training & Evaluation (2 Marks)

- Classification: Accuracy, Precision, Recall, F1
- Regression: MAE, RMSE, R^2
- Correct interpretation and comparison

Leakage Experiment & Analysis (1 Mark)

- Demonstrates incorrect workflow
- Explains what leaked
- Explains performance inflation

Model Comparison & Justification (1 Mark)

- Logical comparison
- Correct reasoning for best model choice

Performance Levels – CLO-2

Level	Marks	Description

Excellent	6–7	Fully correct pipelines, leakage-safe, proper metrics, serialization included
Good	4–5	Minor issues but conceptually correct
Satisfactory	3	Basic pipeline works but weak explanation
Needs Improvement	1–2	Major conceptual or implementation errors
Poor	0	Non-functional or incorrect workflow

CLO-4 (3 Marks – Foundational)

Script-Based Execution (1 Mark)

- Fully runnable .py files
- No notebook dependency

Reproducibility (1 Mark)

- Uses fixed random_state
- Deterministic results
- Clean, consistent preprocessing

Project Structure & Deployment Readiness (1 Mark)

- Clean ZIP structure
- README with execution instructions
- Serialized model artifacts present
- Organized files suitable for containerization later

Performance Levels – CLO-4

Level	Marks	Description
Excellent	3	Clean structure, reproducible, serialization included

Good	2	Mostly structured, minor documentation gaps
Needs Improvement	1	Disorganized but runnable
Poor	0	Non-reproducible or poorly structured

Summary

CLO	Type	What is Assessed
CLO-2	Direct (7)	Complete ML pipelines + leakage prevention + evaluation + serialization
CLO-4	Foundational (3)	Reproducible script-based workflow + deployment readiness